Software Testing – 3

Lecture # 25

Recap and Today's Lecture

- We ended our last lecture on the design of test cases
- This is a very interesting and vast topic and requires through investigation, and we'll start a discussion on this topic in a little while, today
- But before that, let's look at broad categories of software testing

Broad Categories of Testing

- General testing
- Specialized testing
- Testing that involves users or clients

General Forms of Testing

- Concerned with almost any kind of software and seek to eliminate common kinds of bugs such as branching errors, looping errors, incorrect outputs, and the like

Examples of General Forms of Testing

- Subroutine testing
- Unit testing
- System testing of full application
- New function testing
- Regression testing
- Integration testing

Specialized Forms of Testing

- More narrow in focus and seek specific kinds of errors such as problems that only occur under full load, or problems that might slow down performance

Examples of Specialized Forms of Testing

- Viral protection testing
- Stress or capacity testing
- Performance testing
- Security testing
- Platform testing
- Year 2000 testing was also an example of this
- Independent testing

Forms of Testing Involving Users

- The forms of testing involving users are aimed at usability problems and ensuring that all requirements have been in fact implemented

Examples of Forms of Testing Involving Users

- Customer acceptance testing
- Field (Beta) testing
- Usability testing
- Lab testing
- Clean-room statistical testing

- Let’s now focus our attention to the discussion, we were having at the very end of the last lecture, i.e., test case design

Test Case Design - 1

- Any engineered product can be tested in one of two ways
  - Knowing the specified function that a product has been designed to perform
  - Knowing the internal workings of a product

Test Case Design - 2

- In the first case, tests can be conducted that demonstrate each function is fully operational while at the same time searching for errors in each function
- In the second case, tests can be conducted to ensure that internal operations are performed according to the specifications and all internal components have been adequately exercised

Test Case Design - 3

- In the first case, testing is focused on the external behavior of a software system or its various components, and we cannot see inside the components
- While in the second case, testing is focused on the internal implementation, and we must see inside the component

Testing Techniques

- Black-box testing (BBT)
  - aka functional/behavioral testing
- White-box testing (WBT)
  - aka structural/glass-box testing

Black-Box Testing - 1

- Black-box testing alludes to tests that are conducted at the software interface
- Although they are designed to uncover errors, they are also used to demonstrate that software functions are operational, that input is properly accepted and output is correctly produced, and that the integrity of external information is maintained

Black-Box Testing - 2

- A block-box test examines some fundamental aspect of a system with little regard for the internal logical structure of the software

Black-Box Testing - 3

- The inner structure or control flow of the application is not known or viewed as irrelevant for constructing test cases. The application is tested against external specifications and/or requirements in order to ensure that a specific set of input parameters will in fact yield the correct set of output values

Black-Box Testing - 4

- It is useful for ensuring that the software more or less is in concordance with the written specifications and written requirements
- The simplest form of BBT is to start running the software and make observations in the hope that it is easy to distinguish between expected and unexpected behavior

Black-Box Testing - 5

- This is ad-hoc testing and it is easy to identify some unexpected behavior, like system crash
- With repeated executions, we can determine the cause to be related to software and then pass that information to the people responsible for repairs

Black-Box Testing Approaches

- System testing of full application
- New function testing
- Lab testing
- Usability testing
- Customer acceptance testing
- Field (Beta) testing
- Clean-room statistical testing

White-Box Testing - 1

- White-box testing of software is predicated on close examination of procedural detail
- Logical paths through the software are tested by providing test cases that exercise specific sets of conditions and/or loops
- The “status of the programs” may be examined at various points to determine if the expected/asserted status corresponds to the actual status

White-Box Testing - 2

- The test developer is privy to inner structure of the application and knows the control flow through the application, or at least knows the control if the software works correctly
- It is useful for ensuring that all or at least most paths through the applications have been executed in the course of testing

White-Box Testing - 3

- Using white-box testing methods, software engineers can derive test cases that
  - Guarantee that all independent paths within a module have been exercised at least once
  - Exercise all logical decisions on their true and false sides
  - Execute all loops at their boundaries and within their operational bounds
  - Exercise internal data structures to ensure their validity

White-Box Testing - 4

- The simplest form of WBT is statement coverage testing through the use of various debugging tools, or debuggers, which help us in tracing through program executions
- By doing so, the tester can see if a specific statement has been executed, and if the result or behavior is expected

White-Box Testing - 5

- One of the advantages is that once a problem is detected, it is also located
- However, problems of omission or design problems cannot be easily detected through white-box testing, because only what is present in the code is tested
- Another important point is that the tester needs to be very familiar with the code under testing to trace through it executions

White-Box Testing - 6

- Therefore, typically white-box testing is performed by the programmers themselves
- We’ll have some discussion on this topic, that is who is most productive for which kind of testing at a later stage in this course

White-Box Testing Approaches

- Subroutine testing
- Unit testing
- Viral protection testing
- Stress or capacity testing
- Performance testing
- Security testing
- Year 2000 testing

Mixed Testing Approaches

- Independent testing
- Regression testing
- Integration testing
- Platform testing

Comparing BBT with WBT

- It is the perspective, which primarily distinguishes black-box testing from white-box testing
- However, BBT and WBT can be compared using other criteria also

Comparing BBT with WBT with Perspective

- BBT view the objects of testing as a black-box while focusing on testing the input-output relations or external functional behavior; while WBT views the objects as a glass-box where internal implementation details are visible and tested

Comparing BBT with WBT with Objects

- Although the objects tested may overlap occasionally, WBT is generally used to test small objects, such as small software products or small units of large software products; while BBT is generally more suitable for large software systems or substantial parts of them as a whole

Comparing BBT with WBT with Timeline

- WBT is used more in early sub-phases of testing for large software systems, such as unit and component testing; while BBT is used more in late sub-phases, such as system and acceptance testing

Comparing BBT with WBT with Defect Focus - 1

- In BBT, failures related to specific external functions can be observed, leading to corresponding faults being detected and removed. The emphasis is on reducing the chances if encountering functional problems by tagret customers

Comparing BBT with WBT with Defect Focus - 2

- In the WBT, failures related to internal implementation can be observed, leading to corresponding faults being detected and removed directly. The emphasis is on reducing internal faults so that there is less chance for failures later on no matter what kind of application environment the software is subjected to

Comparing BBT with WBT with Defect Detection & Fixing - 1

- Defects detected through WBT are easier to fix than those through BBT because of the direct connection between the observed failures and program units and implementation details in WBT. However, WBT may miss certain type of defects, such as omission and design problems, which could be detected by BBT

Comparing BBT with WBT with Defect Detection & Fixing - 2

- In general, BBT is effective in detecting and fixing problems of interfaces and interactions, while WBT is effective for problems localized within a small unit

Comparing BBT with WBT with Techniques

- Various techniques can be used to build models and generate test cases to perform systematic BBT, and others can be used for WBT, with some of the same techniques being able to be used for both WBT and BBT. A specific technique is a BBT one if external functions are modeled; while the same technique can be a WBT one if internal implementations are modeled

Comparing BBT with WBT with Tester

- BBT is typically performed by dedicated professional testers, and could also be performed by third-party personnel in a setting of IV&V (independent verification and validation); while WBT is often performed by developers themselves

References

- Software Engineering: A Practitioner’s Approach 5th Edition by Roger S. Pressman (Ch 17.2)
- Software Quality: Analysis and Guidelines for Success by Capers Jones
- Software Quality Engineering: Testing, Quality Assurance, and Quantifiable Improvement by Jeff Tian, IEEE Computer Society, 2005 (Chapter 6.3)